

FactoryOps

Product Requirements

1. One-Line Summary

FactoryOps Pharma is an always-on local AI maintenance agent for pharmaceutical production equipment. It detects anomalies, investigates machine documentation and maintenance history, quantifies production and quality risk, and recommends the safest GMP-compliant intervention before equipment failure disrupts the batch.

2. Problem Statement

Pharmaceutical manufacturing technicians lose valuable production time when tablet-press abnormalities occur, because they must manually interpret machine alarms, production parameters, maintenance history, troubleshooting guides, and GMP procedures before deciding whether to adjust, inspect, maintain, or stop the machine. In a regulated environment, that decision also has to respect approved operating limits and be fully documented.

3. Scope

Domain: Solid-dose pharmaceutical manufacturing — rotary tablet-compression line.

Target equipment: A single rotary tablet press, covering the failure modes that account for most unplanned stoppages:

- Abnormal compression force
- Inconsistent tablet weight
- Sticking / picking
- Excessive machine vibration (typically tooling or bearing degradation)

A real production tablet press can produce tens or hundreds of thousands of tablets per hour. Natoli's NP-155, for example, is documented at more than 280,000 tablets per hour and records critical alarms and batch production data^[4].

4. Product

FactoryOps Pharma is not primarily a dashboard. The dashboard is the interface; the product is an agent that continuously monitors tablet-press operating data, detects an abnormal condition, investigates it against local machine documentation, maintenance history, and approved operating procedures, quantifies production and quality risk, and recommends the safest permitted response.

The agent acts, rather than just displays. No one has to notice a problem and ask a question — the agent notices, investigates, explains, quantifies, and recommends on its own, with a policy gate in front of anything requiring approval.

Regulatory fit: FDA recognizes AI/ML for smart monitoring and predictive maintenance in drug manufacturing, including detecting deviations that trigger maintenance and reduce downtime^[1]. Time-critical manufacturing software is expected to stay near the equipment for performance and security reasons^[1] — which is why FactoryOps Pharma runs entirely on-premise rather than depending on a cloud API.

This local-first approach also aligns with current U.S. policy momentum toward strengthening domestic pharmaceutical manufacturing^[7].

5. Technical Documentation (RAG Corpus)

5.1 Machine-specific documentation

- Tablet press operating manual
- Troubleshooting guide — public references exist for Natoli^[5] and Fette^[6] presses covering overload settings, sticking/picking, compression-force problems, ejection and punch issues
- Maintenance SOP
- Machine alarm/fault catalogue

5.2 GMP requirements

- FDA Q7 as the policy/constraint document^[3] — requires preventative-maintenance procedures, calibration of critical equipment, maintenance records, controlled operating ranges, and documented computerized-system changes

5.3 AI/manufacturing guidance

- FDA's discussion paper on AI in drug manufacturing^[1] — covers smart monitoring, predictive maintenance, edge computing, and the governance implications of cloud processing

Sourcing rule: Do not invent manufacturer-specific alarm codes and imply they are real. Use a generic fault code (e.g. TP-VIB-HIGH) or an actual documented code only when the OEM source is confirmed.

6. Input Schema

```
machine_id: TP-04
machine_type: Rotary Tablet Press
manufacturer: Demo/Natoli-class rotary press

batch_id: BATCH-2026-0912-07
product: 500 mg tablet
batch_target_units: 600000

timestamp: 02:14:30
```

```
turret_speed_rpm: 46
main_compression_force_kn: 51.2
precompression_force_kn: 28.4
tablet_weight_mean_mg: 506
tablet_weight_rsd_pct: 4.2

vibration_mm_s: 8.1
motor_temperature_c: 81

alarm_code: TP-VIB-HIGH

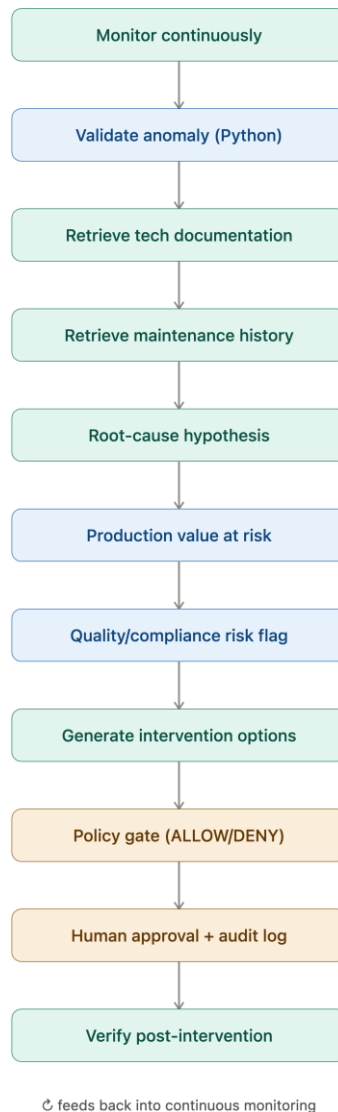
recent_quality_event:
  tablet_weight_out_of_range_pct: 3.8

maintenance_history:
  last_bearing_inspection_days: 172
  prior_similar_incidents: 3

production:
  tablets_per_hour: 180000
  contribution_margin_per_1000_tablets: $X

approved_limits:
  max_vibration_mm_s: 6.0
  max_motor_temperature_c: 75
  tablet_weight_tolerance_pct: +/- 5
```

7. Process — The Closed Loop

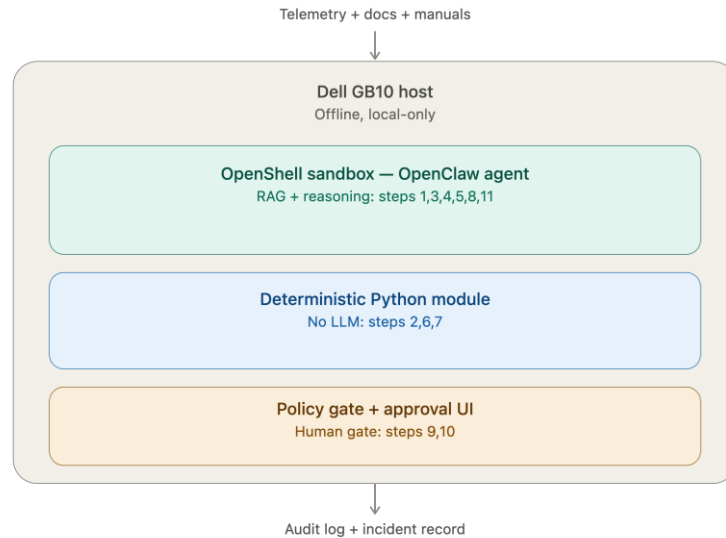


#	Stage	What happens
1	Monitor continuously	Agent polls the local telemetry feed with no human prompt. A vibration trend exceeding the approved threshold automatically opens an incident.
2	Validate the anomaly	Deterministic calculation — not the LLM — computes the deviation from threshold and correlates other signals (temperature, weight variability, throughput).
3	Retrieve technical documentation	Agent searches locally for relevant troubleshooting, degradation, and preventive-maintenance guidance, and cites which sections it used.

4	Retrieve maintenance history	Queries whether this machine has shown the pattern before — prior incident count, outcomes, and typical time from first alert to forced shutdown.
5	Generate root-cause hypothesis	Produces a likely cause with a High / Medium / Low confidence rating and supporting evidence — never a fabricated numeric probability unless a real model backs it.
6	Calculate production value at risk	Deterministic calculation: production rate × expected downtime × contribution margin. Reports both tablets at risk and dollars at risk.
7	Calculate quality/compliance risk	Flags the batch for QA review required when equipment operates outside approved ranges. The agent never autonomously declares a batch safe or unsafe.
8	Generate intervention options	Produces 2–3 ranked options (e.g. continue / reduce speed / controlled shutdown), each with a risk tier and estimated production impact.
9	Policy gate	Deterministic rule engine: out-of-range plus quality impact → DENY autonomous continuation; a change to validated parameters → REQUIRE approval; diagnostic-only → ALLOW.
10	Human approves	A supervisor or QA reviewer approves the action; the system logs incident ID, operator, timestamp, evidence, retrieved documentation, proposed intervention, and approval.
11	Verify after intervention	Confirms post-maintenance telemetry has returned to approved range; the batch remains flagged for QA disposition until formally cleared.

The loop then returns to Stage 1 — continuous monitoring never stops.

Here's where that pipeline needs to physically live on the Dell GB10, using the components already defined in the setup guide:



8. Output

FACTORYOPS PHARMA

Incident TP-04-0912

STATUS

CRITICAL EQUIPMENT ANOMALY

LIKELY CAUSE

Main-drive bearing degradation

Confidence: HIGH

EVIDENCE

- Vibration +35% above approved limit
- Motor temperature +6C above limit
- 3 similar historical incidents
- Tablet-weight variability increasing

BUSINESS IMPACT

720,000 tablets potentially affected

\$25,200 production contribution at risk

QUALITY IMPACT

Current batch requires QA review

RECOMMENDED ACTION

Controlled shutdown

Inspect main-drive bearing and alignment

Estimated intervention: 3.5 hours

POLICY

Human approval required

9. Key Design Principles

- Deterministic where it matters. Anomaly validation, production impact, and quality-risk flagging are computed programmatically — never left to the LLM to estimate or invent.
- Confidence bands, not fake precision. Root-cause hypotheses use High/Medium/Low confidence unless a defensible probability model exists.
- The agent recommends; it doesn't decide. Batch safety and any action affecting validated parameters require human/QA sign-off. Regulation is treated as an asset, not a burden.

Everything is auditable. Every incident produces a complete record — evidence, retrieved sources, proposed action, approval — consistent with GMP's emphasis on equipment, maintenance, and computerized-system records^[3].

- Local by design. All inference runs on-premise, with no cloud API in the runtime path. This is both the regulatory story (sensitive plant and batch data never leaves the site) and the latency story (real-time anomaly response, not batch processing).

10. Ownership

Owner	Responsibility
Farhan	Telemetry and anomaly engine, deterministic calculations (thresholds, production-at-risk, maintenance-history lookup, incident/audit log)
Ferdi	Local agent, RAG, and orchestration (document ingestion/retrieval, root-cause reasoning, tool calling)
Gio	Policy gate and approval interface (rule engine, ALLOW/DENY/HUMAN APPROVAL logic, UI, intervention options, approval workflow)
Tika	Business and domain framing (workflow definition, ROI model, revenue framework, product narrative)

11. References

[1] FDA, "Artificial Intelligence in Drug Manufacturing" (discussion paper covering smart monitoring, predictive maintenance, and edge computing) — <https://www.fda.gov/media/165743/download>

[2] FDA, discussion paper on AI in drug manufacturing — governance considerations for cloud vs. local processing — <https://www.fda.gov/media/165743/download>

[3] FDA, "Q7 Good Manufacturing Practice Guidance for Active Pharmaceutical Ingredients" (ICH Q7) — <https://www.fda.gov/regulatory-information/search-fda-guidance-documents/q7->

[good-manufacturing-practice-guidance-active-pharmaceutical-ingredients-guidance-industry](#)

[4] Natoli Engineering, NP-155 rotary tablet press — product specifications —

<https://natoli.com/products/tablet-presses/production-tablet-presses/np-155/>

[5] Natoli Engineering, tablet press troubleshooting resources (sticking, picking, overload, downtime) —

<https://natoli.com/do-you-need-help-with-tablet-press-troubleshooting/>

[6] Fette Compacting, tablet press pocket guide — operation and troubleshooting reference —

<https://www.scribd.com/doc/109577279/Fette-Compression-Machine-pocketguide>

[7] FDA, "Actions to Support and Strengthen Domestic Drug Manufacturing" —

<https://www.fda.gov/industry/fda-actions-support-and-strengthen-domestic-drug-manufacturing>

Initial Draft

Scope -> **pharmaceutical manufacturing**, specifically a **tablet-compression line**

Why: it preserves the strongest parts of FactoryOps—local/on-premise data, machine diagnostics, maintenance documentation, always-on monitoring, hard-dollar downtime/quality impact—while giving you much better public technical documentation than defense. FDA explicitly recognizes AI/ML for smart monitoring and maintenance in drug manufacturing, including detecting deviations that trigger maintenance and reduce process downtime. ([U.S. Food and Drug Administration](#)) FDA also notes that time-critical manufacturing software may need to stay near equipment for performance/security reasons, which is a very clean GB10 justification. ([U.S. Food and Drug Administration](#))

Your current requirement document already defines the right closed loop—sensor anomaly → investigation → document retrieval → root cause → production impact → policy gate → recommendation → verification. I would just make that loop domain-specific.

Proposed FactoryOps scope

Problem statement:

Pharmaceutical manufacturing technicians lose valuable production time when tablet-press abnormalities occur because they must manually interpret machine alarms, production parameters, maintenance history, troubleshooting guides, and GMP procedures before deciding whether to adjust, inspect, maintain, or stop the machine.

In a regulated environment, the decision also has to respect approved operating limits and be documented.

Scope:

Solid-dose pharmaceutical manufacturing — rotary tablet-compression line.

For the hackathon, narrow further to **one rotary tablet press** and 3–4 failure scenarios, such as:

- abnormal compression force
- inconsistent tablet weight
- sticking/picking
- excessive machine vibration / likely tooling or bearing issue

This is better than “pharma factory” broadly because the inputs, failure modes, documents, calculations, and outputs become concrete.

A real production tablet press can produce tens or hundreds of thousands of tablets per hour; Natoli’s NP-155, for example, lists configurations ranging roughly from 42,000 to 270,000 tablets/hour and records critical alarms/error codes and batch production data. (natoli.com)

Product:

FactoryOps Pharma — an always-on local maintenance and production-risk agent for pharmaceutical manufacturing equipment.

It is **not primarily a dashboard**.

The dashboard is the interface.

The product is an **agent** that:

continuously monitors tablet-press operating data, detects an abnormal condition, investigates it against local machine documentation + maintenance history + approved operating procedures, quantifies production/quality risk, and recommends the safest permitted response.

That stays aligned with your requirement that FactoryOps “acts, rather than just displays.”

Technical documents

I would use three document classes.

1. Machine-specific troubleshooting documentation

For the demo, a rotary tablet press is unusually convenient because public troubleshooting material exists.

Natoli tablet-press technical resources cover operating problems such as overload settings, sticking/picking, replacement parts, and maintaining presses with minimal downtime. (natoli.com)

A public Fette tablet-press reference guide also contains actual troubleshooting guidance—for example machine-start failures, compression-force problems, ejection issues, punch issues, and control parameters. ([Scribd](https://www.scribd.com/document/354444444/Fette-Compacting-Tablet-Press-Reference-Guide))

For your RAG corpus, ideally use:

- tablet press operating manual
- troubleshooting guide
- maintenance SOP
- machine alarm/fault catalogue

2. FDA / GMP manufacturing requirements

Use FDA Q7A as the policy/constraint document.

It specifically requires preventative-maintenance procedures, calibration of critical equipment, maintenance records, controlled operating ranges, and documented computerized-system changes. ([U.S. Food and Drug Administration](https://www.fda.gov/oc/ohrt/q7a-guidance))

Useful link:

[FDA Q7A GMP Guidance](https://www.fda.gov/oc/ohrt/q7a-guidance)

3. AI/manufacturing guidance

FDA's drug-manufacturing AI paper directly discusses smart monitoring, predictive maintenance, edge computing, data management, and the governance complications created by cloud processing. ([U.S. Food and Drug Administration](https://www.fda.gov/oc/ohrt/q7a-guidance))

[FDA — Artificial Intelligence in Drug Manufacturing](https://www.fda.gov/oc/ohrt/q7a-guidance)

That is especially useful for your “**Why GB10?**” slide.

Input

I would make the event schema something like:

machine_id: TP-04
machine_type: Rotary Tablet Press
manufacturer: Demo/Natoli-class rotary press

batch_id: BATCH-2026-0912-07
product: 500 mg tablet
batch_target_units: 600,000

timestamp: 02:14:30

turret_speed_rpm: 46
main_compression_force_kn: 51.2
precompression_force_kn: 28.4
tablet_weight_mean_mg: 506
tablet_weight_rsd_pct: 4.2

vibration_mm_s: 8.1
motor_temperature_c: 81

alarm_code: TP-VIB-HIGH

recent_quality_event:
 tablet_weight_out_of_range_pct: 3.8

maintenance_history:
 last_bearing_inspection_days: 172
 prior_similar_incidents: 3

production:
 tablets_per_hour: 180000
 contribution_margin_per_1000_tablets: \$X

approved_limits:

```
max_vibration_mm_s: 6.0
max_motor_temperature_c: 75
tablet_weight_tolerance_pct: +/- 5
```

For the hackathon, these values can be **synthetic**, as long as you clearly label them as simulated plant data.

Do not invent manufacturer-specific alarm codes and imply they are real. Either use a generic demo fault such as TP-VIB-HIGH or use an actual documented code only when you have the OEM source.

Process

I would make this the exact pipeline.

Step 1 — Monitor continuously

FactoryOps polls the local telemetry/event feed.

No human prompt.

```
TP-04 vibration:
4.2 → 4.6 → 5.3 → 6.7 → 8.1 mm/s
```

Threshold is exceeded.

Agent automatically opens an incident.

Step 2 — Validate the anomaly

Don't ask the LLM whether something is abnormal.

Use deterministic Python.

Example:

```
current_vibration = 8.1
approved_threshold = 6.0
```

```
deviation = +35%
```

Then correlate other signals:

temperature above threshold = YES
weight variability increasing = YES
throughput declining = YES

Step 3 — Retrieve relevant technical documentation

Search locally for:

- vibration troubleshooting
- bearing degradation
- compression instability
- abnormal motor temperature
- preventive maintenance procedure

The agent should cite which sections it used.

For example:

Retrieved:

1. Tablet Press Troubleshooting Guide – Vibration
2. Maintenance SOP – Main Drive/Bearing Inspection
3. GMP Maintenance Procedure – equipment intervention requirements

This directly improves on your current research brief's simpler “technician searches manuals” concept. Your brief correctly identifies the open competitive space as **on-premise judgment**, not physical automation.

Step 4 — Retrieve maintenance history

Query:

Has TP-04 had this pattern before?

Example result:

3 similar incidents.

2 resulted in bearing replacement.
1 resulted in alignment correction.

Mean time from first vibration alert to forced shutdown:
11.4 hours.

This is where the agent becomes better than simply searching a PDF.

Step 5 — Generate a root-cause hypothesis

Example:

Likely cause:

Main-drive bearing degradation.

Confidence: HIGH

Evidence:

- + vibration 35% above approved threshold
- + motor temperature above operating range
- + similar historical pattern
- + troubleshooting documentation identifies bearing/alignment as likely causes

I would avoid presenting “81% probability” unless you actually have a model producing 81%. Your existing requirement doc currently uses 81% as an illustrative output and explicitly notes that it still needs to become reproducible.

Use:

High / Medium / Low confidence

unless you build a defensible probability model.

Step 6 — Calculate production value at risk

Again: Python, not LLM.

For example:

```
production_rate = 180,000 tablets/hour
expected_downtime = 4 hours
contribution_margin = $35 / 1,000 tablets
```

Then:

$180 \times \$35 \times 4$
= \$25,200 production contribution at risk

You can also calculate:

tablets at risk = 720,000

For pharma, I would actually display **both**.

720,000 tablets potentially affected

\$25,200 production contribution at risk

The first feels industry-specific.

The second gives judges the Dell revenue-framework metric.

Step 7 — Calculate quality/compliance risk

This is what makes pharma materially more differentiated from generic manufacturing.

Example:

Batch BATCH-07 currently running.

Potential impact:

- tablet weight variability trending upward
- equipment outside approved vibration range
- current batch may require QA review if machine condition affects critical quality attributes

Do **not** have the agent autonomously declare a batch safe or unsafe.

Instead:

QUALITY REVIEW REQUIRED

FDA guidance requires equipment to operate in qualified ranges and deviations affecting product quality to be investigated/documented. ([U.S. Food and Drug Administration](#))

Step 8 — Generate possible interventions

For example:

OPTION A

Continue operation

Risk: HIGH

Not recommended

OPTION B

Reduce turret speed 46 → 35 RPM

Risk: MEDIUM

Estimated output loss: 24%

Allows production until planned intervention

OPTION C

Controlled shutdown + bearing inspection

Risk: LOW

Expected maintenance: 3.5 hr

Recommended

This is more impressive than simply:

“Replace bearing.”

You are showing operational decision support.

Step 9 — Policy gate

Now Gio's deterministic gate matters.

Example rules:

IF equipment exceeds qualified operating range

AND product quality may be affected

THEN autonomous continuation = DENY

IF intervention changes validated production parameters

THEN QA/supervisor approval = REQUIRED

IF action is diagnostic-only

THEN autonomous action = ALLOW

Output:

PROPOSED ACTION

Controlled shutdown + inspection

POLICY STATUS

HUMAN APPROVAL REQUIRED

Required approvers:

Maintenance Supervisor

QA

This uses regulation as an asset rather than a burden.

Step 10 — Human approves

Button:

Approve Controlled Shutdown

Then FactoryOps logs:

Incident ID

Operator

Timestamp

Evidence

Documentation retrieved

Proposed intervention

Approval

FDA GMP guidance puts significant emphasis on equipment-use, maintenance, calibration, and computerized-system records, so an audit trail is especially credible in this vertical. ([U.S. Food and Drug Administration](#))

Step 11 — Verify after intervention

Inject synthetic post-maintenance telemetry:

vibration: 8.1 → 4.3 mm/s

temperature: 81 → 66°C

tablet weight RSD: 4.2 → 1.1%

Agent:

INCIDENT RESOLVED

Machine returned to normal operating range.

Maintenance recommendation:

Inspect bearing at next scheduled service.

QA review:

Batch remains flagged for QA disposition.

Now you have the full always-on closed loop that your requirements document originally envisioned.

Output

The final screen could be:

FACTORYOPS PHARMA

Incident TP-04-0912

STATUS

⚠ CRITICAL EQUIPMENT ANOMALY

LIKELY CAUSE

Main-drive bearing degradation

Confidence: HIGH

EVIDENCE

- Vibration +35% above approved limit
- Motor temperature +6°C above limit
- 3 similar historical incidents
- Tablet-weight variability increasing

BUSINESS IMPACT

720,000 tablets potentially affected

\$25,200 production contribution at risk

QUALITY IMPACT

Current batch requires QA review

RECOMMENDED ACTION

Controlled shutdown

Inspect main-drive bearing and alignment

Estimated intervention: 3.5 hours

POLICY

Human approval required

Maintenance Supervisor + QA

[APPROVE] [REVIEW EVIDENCE]

That's much stronger than a generic FactoryOps screen.

Pharma vs defense manufacturing

For **this hackathon**, I would pick pharma even before completing TAM research.

	Pharma manufacturing	Defense manufacturing
Hard local/on-prem need	4/5	5/5
Always-on machinery	5/5	5/5
Public technical documents	5/5	2/5
Regulatory/policy documents	5/5	4/5
Hard-dollar value	5/5	5/5
Demo realism	5/5	3/5
Ability to validate claims publicly	5/5	2/5
Risk of overclaiming	Low-medium	High
Hackathon buildability	High	Medium

Defense does have an exceptionally good local-first thesis. In fact, a U.S. Air Force SBIR has explicitly sought an **on-prem AI/ML maintenance platform** with an LLM knowledge base for guided troubleshooting of critical depot industrial plant equipment. ([U.S. Department of War](#))

That's almost FactoryOps.

But that's also precisely the problem: it would be harder to obtain authentic equipment data, manuals, fault histories, or representative workflows without either inventing them or wandering into sensitive territory.

Pharma gives you:

regulated + local + documentation-heavy + machine-intensive + measurable downtime + public source material.

And the market has current policy momentum: FDA launched initiatives in 2026 specifically aimed at strengthening domestic pharmaceutical manufacturing and advanced manufacturing. ([U.S. Food and Drug Administration](#))

So my proposed final one-liner is:

FactoryOps Pharma is an always-on local AI maintenance agent for pharmaceutical production equipment that detects anomalies, investigates machine documentation and maintenance history, quantifies production and quality risk, and recommends the safest GMP-compliant intervention before equipment failure disrupts the batch.

OWNERSHIP

Owner	Final ownership	What they should actually build
Farhan	Telemetry + anomaly engine + deterministic calculations	Synthetic tablet-press event stream, thresholds, anomaly detection, production-at-risk calculation, maintenance-history lookup, incident/audit log
Ferdi	Local agent + RAG + orchestration	NemoClaw/OpenClaw/OpenShell, local model, ingestion/retrieval of machine manual + GMP/SOP docs, root-cause reasoning, tool calling
Gio	Policy gate + approval UX + product interface	GMP rule engine, ALLOW / DENY / HUMAN APPROVAL logic, Streamlit/UI, intervention options, approval workflow
Tika	Business + domain + pitch	Pharma workflow, customer/problem, ROI assumptions, revenue framework, what the agent should recommend, demo storyline, slides/pitch

